

Guardrails: SCPs, RCPs and Policy Evaluation

Five policy types decide every request. This is the order they run in.

A service control policy never grants anything. It only **defines the ceiling** that grants must fit under.

The five policy types that can affect one request

- ◆ **Service control policy.** Attached in Organizations. Caps what IAM principals in an account may do.
- ◆ **Resource control policy.** Also in Organizations. Caps access **to** that account's resources, whoever asks.
- ◆ **Identity-based policy.** On a user, group or role. One of the two places a grant can come from.
- ◆ **Resource-based policy.** On the resource itself. The other place a grant can come from.
- ◆ **Permissions boundary** and **session policy.** Ceilings on one principal, never grants.
- ◆ Only two of the five can say yes. The other three can only say no.

ON THE EXAM

Two granters, three limiters. A limiter offered as the way to give access is always wrong.

How AWS actually decides

01

Explicit deny?

Any deny in any policy. Stop. Denied.

02

SCP allows?

Action must be inside the SCP ceiling.

03

RCP allows?

Resource must be reachable under the RCP.

04

Boundary allows?

Action must be inside the boundary.

05

Grant exists?

Identity or resource policy must allow it.

ON THE EXAM

One explicit deny anywhere ends the evaluation. Everything else must all agree before a request succeeds.

Think of it as **four gates and one key**. The gates can only be shut. Only the key opens anything.

SCP or RCP?

SERVICE CONTROL POLICY

Caps what your principals do

- Follows the identity making the request
- Applies to IAM users and roles here
- Skips the management account
- Use: deny unapproved Regions

RESOURCE CONTROL POLICY

Caps access to your resources

- Follows the resource being touched
- Applies to any caller, even outsiders
- Also skips the management account
- Use: stop a bucket being shared out

THE TIEBREAKER

Ask who the scenario is worried about. Our people doing the wrong thing means SCP. Outsiders reaching our data means RCP.

The management account exception

- ◆ **SCPs and RCPS do not apply to the management account.** Attach one at the root and it is still ignored.
- ◆ This is why the guidance is to run **no workloads** in the management account.
- ◆ It is also why sensitive functions get **delegated administrator** accounts instead.
- ◆ Service-linked roles are exempt too, so AWS services keep working under a restrictive policy.

ON THE EXAM

Nothing you attach in Organizations constrains the management account. Treat it as outside the guardrails, always.

Permissions boundary or session policy?

PERMISSIONS BOUNDARY

A durable ceiling on one principal

- Attached to a user or role and stays there
- Survives across every session
- Classic use: let developers create roles safely
- The delegation pattern the exam tests most

SESSION POLICY

A temporary ceiling on one session

- Passed at AssumeRole time, in the API call
- Applies only to that set of credentials
- Classic use: one broad role, narrowed per customer
- Common in SaaS multi-tenant designs

THE TIEBREAKER

Permanent limit on a principal means boundary. Limit that varies per session means session policy.

Cross-account access needs two yeses

REQUEST	WHAT MUST ALLOW IT	THE COMMON MISTAKE
Same account, identity policy	Identity policy alone	Resource policy is optional here
Same account, resource policy	Resource policy alone	Either one suffices in-account
Cross-account: S3, KMS, SQS	Both sides	Assuming the resource policy is enough
Cross-account via AssumeRole	Trust policy and caller	Forgetting sts:AssumeRole on the caller
Cross-account with SCPs	Both accounts' SCPs	Checking only their own account

ON THE EXAM

In-account, either policy suffices. Across accounts, both sides must say yes.

Where SCPs are actually placed

- ◆ **Policies inherit downward and intersect.** An action must be allowed at root, OU and account.
- ◆ Inheritance is not a union. A narrow policy above cannot be widened below.
- ◆ AWS attaches **FullAWSAccess** by default. Detach it and you have denied everything.
- ◆ Common design: **allow-list at the root, deny-list at the OU**, so exceptions sit near the workloads.

ON THE EXAM

An action must survive every level. The effective permission is the intersection, never the union.

An SCP that **allows** a service does nothing on its own. Someone still has to be granted the permission.

Reading the scenario for the answer

THE SCENARIO SAYS

Prevent any account from doing X, ever

Stop data being shared outside the organisation

Let developers create roles but not escalate

One role, different limits per tenant

Third party needs access to our account

The control did not apply to one account

IT IS POINTING AT

SCP with an explicit deny

RCP, or aws:PrincipalOrgID

Permissions boundary

Session policy at AssumeRole

Cross-account role with external ID

That account is the management account

ON THE EXAM

Most policy questions are decided by one phrase. Spot the phrase rather than reasoning it out under time pressure.

Condition keys the exam expects you to know

- ◆ **aws:PrincipalOrgID** – the caller belongs to your organisation. The cleanest way to scope a resource policy without listing accounts.
- ◆ **aws:PrincipalOrgPaths** – narrower still. The caller sits under a specific OU.
- ◆ **aws:SourceArn** and **aws:SourceAccount** – prevent the confused deputy when a service calls on someone's behalf.
- ◆ **aws:RequestedRegion** – the Region restriction pattern, used in almost every landing zone.
- ◆ **aws:ResourceOrgID** – the resource belongs to your organisation. Blocks staff sending data to a personal account.

ON THE EXAM

PrincipalOrgID controls who reaches in. ResourceOrgID controls where your people can send things out.

Meridian Freight has 140 accounts under AWS Organizations. A compliance rule states that no workload may run outside eu-west-1 and eu-west-2. The security team attaches an SCP at the organisation root denying all actions when `aws:RequestedRegion` is not one of those two Regions.

A week later, an auditor finds EC2 instances running in us-east-1 in one account. The SCP is confirmed as attached at the root and correctly written.

What is the MOST likely explanation?

- ☐ A SCPs take up to 24 hours to propagate to all accounts
- ☐ B The account is the organisation's management account
- ☐ C The instances were launched before the SCP was attached, and SCPs are not retroactive
- ☐ D A permissions boundary in that account overrides the SCP

◆ PAUSE AND DECIDE

What is the MOST likely explanation?

- A

 Propagation delay
SCP changes take effect in seconds, and a week has passed regardless. This option invents a limitation that does not exist.
- B

 It is the management account
SCPs never apply to the management account, however they are attached. This is the only explanation consistent with a correct policy at the root.
- C

 Not retroactive
True but irrelevant. Existing instances would keep running, but the scenario says instances are running, not that they were launched recently. And a genuinely blocked account could not have launched them at all.
- D

 Boundary overrides SCP
Backwards. A boundary is another ceiling, not an override. Nothing below the SCP can widen it.

THE SIGNAL

Correct policy, correct attachment, one account unaffected. That combination has exactly one cause.

A platform team wants application teams to create their own IAM roles for their workloads, so the platform team is no longer a bottleneck. Security requires that a team can never create a role with more privilege than the team itself holds.

The company already uses SCPs at the OU level to restrict which services each business unit may use.

What should the platform team implement?

- ☐ A Tighten the OU's SCP to deny iam:CreateRole for all application teams
- ☐ B Grant iam:CreateRole but require that created roles carry a permissions boundary, enforced with a condition
- ☐ C Grant iam:CreateRole and pass a session policy when application teams sign in
- ☐ D Grant iam:CreateRole and review created roles daily with IAM Access Analyzer

◆ PAUSE AND DECIDE

What should the platform team implement?

- A

 Deny role creation
Solves the security requirement by abandoning the business requirement. The whole point was to remove the bottleneck.
- C

 Session policy at sign-in
Limits the team's own session. It places no limit on roles they create, which are separate principals with their own permissions.
- B

 Require a boundary on created roles
The canonical safe-delegation pattern. The condition means a role cannot be created at all unless it carries the boundary, so escalation is structurally impossible.
- D

 Detective review
Finds the escalation after it has happened. The requirement says never, which is a preventive word.

THE SIGNAL

Delegate creation, constrain the creation. Boundary plus a condition requiring it is the pattern AWS documents for exactly this.

A data science account must read objects from an S3 bucket in a separate production account. An administrator adds a bucket policy in the production account allowing the data science role's ARN to perform s3:GetObject.

Calls still fail with an access denied error. The role has the AmazonS3ReadOnlyAccess managed policy attached, and no SCP in either account restricts S3.

What is the MOST likely cause?

- ☐ A The bucket policy alone is insufficient; the identity policy must also allow the specific bucket
- ☐ B The bucket's KMS key policy does not grant the data science role decrypt permission
- ☐ C Cross-account S3 access requires AWS RAM to share the bucket first
- ☐ D The role needs sts:AssumeRole on the production account

◆ PAUSE AND DECIDE

What is the MOST likely cause?

A Identity policy also required

True as a rule, but the scenario already satisfies it. AmazonS3ReadOnlyAccess allows GetObject on every bucket, so the caller's side is covered.

C Share via AWS RAM

RAM shares things like subnets, Transit Gateways and Route 53 rules. S3 buckets are not RAM shareable, and never were.

B KMS key policy missing the role

With both S3 policies satisfied, the remaining gate is the key. A bucket encrypted with a customer managed key needs the key policy to permit Decrypt for the cross-account role.

D sts:AssumeRole needed

Only if the design used role assumption. This design grants the role directly in the bucket policy, so there is no role to assume.

THE SIGNAL

When both S3 policies are correct and access still fails, the encryption key is the next gate. KMS is a separate authorisation decision.

A regulated company must guarantee that data in its S3 buckets can never be accessed by any AWS principal outside its own organisation, across 200 accounts. Buckets are created continuously by application teams.

A previous attempt used a bucket policy template with an `aws:PrincipalOrgID` condition, but audits keep finding buckets created without it.

What should the solutions architect recommend?

- ☐ A An SCP denying `s3:PutBucketPolicy` unless the policy contains `aws:PrincipalOrgID`
- ☐ B A resource control policy at the organisation root denying S3 access where `aws:PrincipalOrgID` does not match
- ☐ C S3 Block Public Access enforced organisation-wide
- ☐ D An AWS Config rule that flags non-compliant bucket policies for remediation

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

 SCP on PutBucketPolicy
Constrains how policies are written, not what access is permitted. Buckets with no policy at all, or access granted by other means, slip straight past.
- C

 Block Public Access
Blocks anonymous and public access only. A named principal in an external AWS account is neither, so the stated risk remains.
- B

 RCP at the root
Applies to every bucket in every account automatically, including ones created tomorrow, and it caps access regardless of what the bucket policy says.
- D

 Config rule
Detective. The requirement says never, and buckets appear continuously, so there is always a window between creation and remediation.

THE SIGNAL

This is exactly what RCPs were built for. Per-bucket policy templates depend on discipline; an RCP does not.

A SaaS provider runs one shared application role that can read from any customer's DynamoDB table. Each customer's data is partitioned by a leading key value. The provider must guarantee a request handled for customer A can never read customer B's items.

There are 4,000 customers and the number grows weekly. Creating a role per customer has been ruled out as unmanageable.

Which approach meets the requirement?

- ☐ A Attach a permissions boundary to the shared role limiting it to one customer's keys
- ☐ B Pass a session policy at AssumeRole scoping dynamodb:LeadingKeys to that customer
- ☐ C Add an SCP to the account restricting DynamoDB access by partition key
- ☐ D Apply a resource-based policy on the table for each customer

◆ PAUSE AND DECIDE

Which approach meets the requirement?

- A

Boundary on the shared role

A boundary is fixed to the principal. One shared role can carry only one boundary, so it would pin every request to the same single customer.
- B

Session policy per assumption

The scope changes per session, which is exactly what varies here. Each set of credentials is limited to one customer's leading keys.
- C

SCP by partition key

SCPs are account-wide ceilings. They cannot vary per request, and would apply identically to all four thousand customers.
- D

Resource policy per customer

DynamoDB has no resource-based policy of the kind described, and 4,000 entries would exceed policy size limits even if it did.

THE SIGNAL

The limit changes per request while the principal stays the same. Only a session policy varies at that granularity.

A third-party monitoring vendor needs read access to a company's CloudWatch metrics. The vendor asks the company to create an IAM role trusting the vendor's AWS account. The company's security team is concerned that the vendor serves many customers, and that another of the vendor's customers could be tricked into causing access to this company's account.

What should the security team require?

- ☐ A An external ID in the role's trust policy, supplied by the company
- ☐ B An IAM user with long-lived access keys instead of a role
- ☐ C An SCP denying all actions from principals outside the organisation
- ☐ D A permissions boundary on the vendor's role limiting it to CloudWatch

◆ PAUSE AND DECIDE

What should the security team require?

- A

 External ID in the trust policy
The documented answer to the confused deputy problem. The vendor must present a secret only this company knows, so another customer cannot cause access here.
- B

 Long-lived access keys
Replaces temporary credentials with permanent ones handed to a third party. Strictly worse, and does not address confusion between the vendor's customers at all.
- C

 SCP denying external principals
SCPs constrain principals in your accounts, not callers from outside. It would also block the access the company wants to grant.
- D

 Boundary limiting to CloudWatch
Correctly limits scope, and is good practice, but scope was never the concern. The concern is who can trigger the assumption.

THE SIGNAL

Third party, many customers, risk of the wrong one triggering access. That is the confused deputy, and external ID is its named fix.

What to carry out of this module

- ◆ **Four gates and one key.** Deny, SCP, RCP and boundary can only shut. Only a grant opens.
- ◆ **SCPs follow the identity, RCPs follow the resource.** Only an RCP stops an outsider.
- ◆ Nothing in Organizations constrains the management account. Run nothing there.
- ◆ Boundary for a permanent limit. Session policy when the limit changes per session.
- ◆ **Never** and **prevent** are preventive words. They eliminate detective options.

ON THE EXAM

This chain reappears in every domain. When an access question looks ambiguous, walk the gates in order rather than guessing.